

# 08 — Latency Hiding, MLP e Little's Law na GPU

Pré-requisito: arquivos 05 e 11 da Phase 1 e [03 — scheduling](#).

## 1. A GPU não torna VRAM rápida

A medição deste lab encontrou ~289,7 ns por dependent load fora de cache. A estratégia da GPU é manter outros warps prontos:

```
warp A: emite load ———— espera ———— volta
warp B:           calcula
warp C:           emite/calcula
warp D:           calcula
```

O scheduler não remove a latência de A; cobre parte dela com trabalho útil de B/C/D.

## 2. Três quantidades diferentes

- **Little's Law:** concorrência média necessária para um throughput-alvo.
- **MLP exposto:** loads independentes que código/threads conseguem manter pendentes.
- **Warps residentes/eligible:** fontes de instruções e pedidos, não pedidos em si.

Uma thread pode ter zero, uma ou várias operações de memória pendentes; um warp pode emitir transações diferentes conforme coalescing. Por isso não iguale "2764 requests" a "2764 threads" de modo literal.

Com 305,21 GB/s, setores de 32 B e 289,7 ns:

```
requests/s ≈ 305,21e9 / 32
L ≈ requests/s × 289,7e-9 ≈ 2764 requests em voo
```

É uma ordem de grandeza útil, não um occupancy calculator.

## 3. Como latency hiding falha

- poucos blocks/warps por grid;
- register/shared pressure reduz residência;
- cadeias de loads dependentes reduzem MLP;
- divergência/barreiras deixam poucos warps eligible;
- todos os warps chegam juntos ao mesmo stall;
- acesso não coalesced aumenta trabalho do subsistema de memória.

## Pergunta rápida

Occupancy de 100% garante os ~2764 pedidos pendentes necessários?

► Resposta

## O que guardar

- Latency hiding troca “esperar parado” por “executar outro warp”.
- Occupancy habilita, mas não mede MLP nem performance.
- Coalescing e concorrência atacam problemas diferentes: eficiência por transação e número de transações em progresso.

**Conexão:** branches divergentes podem reduzir lanes úteis e alterar quais warps ficam prontos; esse é o próximo mecanismo.